TÉCNICO
UNIVERSIDADE
DE LISBOA

Performance Evaluation and Dimensioning of Networks and Systems

METI

Lab 2 Report [EN]

Authors:

José Oliveira (103771) jose.novais.oliveira@tecnico.ulisboa.pt
Afonso Mendes (116024) afonsofmendes@tecnico.ulisboa.pt
Tiago Videira (116069) tiago.g.videira@tecnico.ulisboa.pt

2026/2027 – 1st Semester

Contents

1 Introduction **2**

2 A. Discrete-time Markov chains **3**
2.1 Exercise 1: Bernoulli and 2-DTMC sample paths . . . 3
2.2 Exercise 2: Estimating transitions from the attack trace . . . 4
2.3 Exercise 3: Slotted ALOHA . . . 4
2.3.1 a. Theoretical throughput . . . 4
2.3.2 b. Discrete-event simulation . . . 6
2.3.3 c. ALOHA vs. CSMA, CSMA/CD and CSMA/CA . . . 7

3 B. Discrete-time Markov chains and Continuous-time Markov chains **8**
3.1 Exercise 4: Limiting probabilities of a DTMC . . . 8
3.2 Exercise 5: Simulation of a CTMC . . . 10

4 C. Queuing systems **11**
4.1 Exercise 6: Stability of the M/M/1 queue . . . 11
4.2 Exercise 7: Point-to-point link (`ppl.R`) . . . 12
4.2.1 a. Poisson arrivals, exponential packet sizes . . . 12
4.2.2 b. Poisson arrivals, fixed packet sizes . . . 12
4.2.3 c. Strict priority scheduling . . . 13

5 D. Workloads **14**

6 Conclusion **14**

1 Introduction

This report presents the solutions to the three exercises assigned to class 5 in Laboratory 2 of the Performance Evaluation and Dimensioning of Networks and Systems (DDRS) course. It follows the objective–approach–results–conclusion structure used in the Laboratory 1 report.

2 A. Discrete-time Markov chains

2.1 Exercise 1: Bernoulli and 2-DTMC sample paths

Objective: Compare Bernoulli processes with two-state discrete-time Markov chains (2-DTMCs) using two parameter pairs that have the same stationary mean but markedly different temporal behavior.

Approach: The supplied helper `dtmc_bernoulli_plot.R` uses the transition matrix

$$P = \begin{bmatrix} \alpha & 1-\alpha \\ \beta & 1-\beta \end{bmatrix},$$

The stationary probability of state 1 is

$$\pi_1 = \frac{1-\alpha}{1-\alpha+\beta}.$$

For $(\alpha, \beta) = (0.9, 0.1)$, the helper generates 200 DTMC samples and an independent Bernoulli process with success probability $\pi_1 = 0.5$, plots both paths, and saves them together as `p3.jpeg`.

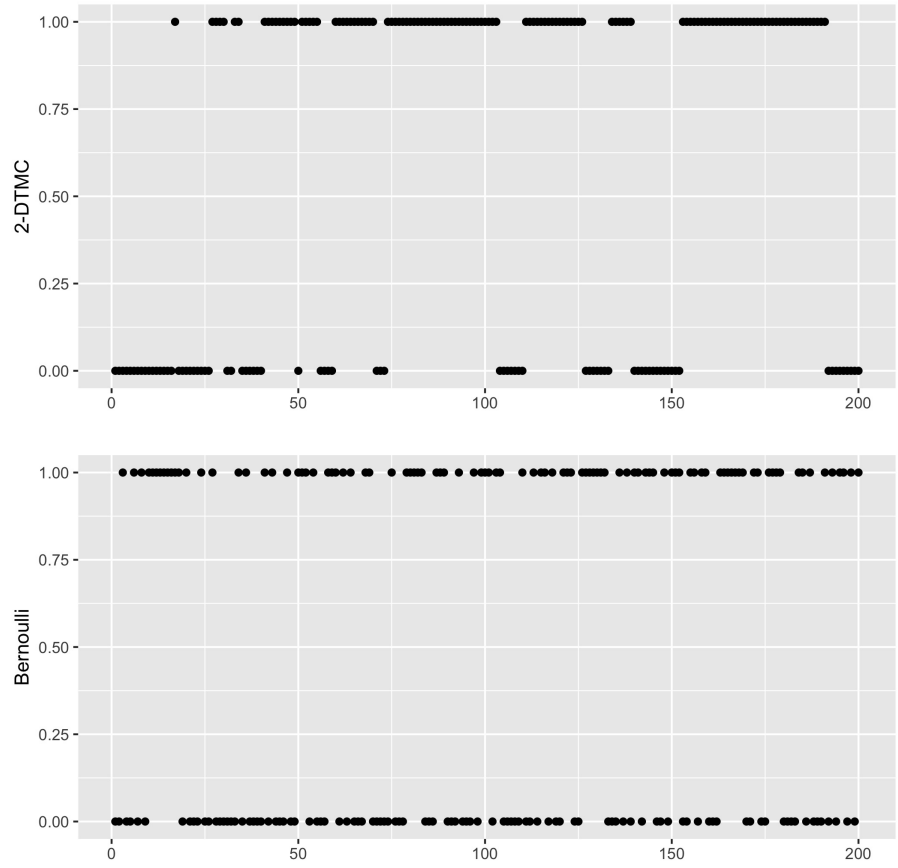


Figure 1: Sample paths from the 2-DTMC and Bernoulli process generated by the supplied helper.

Results and conclusion: Since $\alpha = 0.9$, the DTMC tends to remain in its current state, producing longer runs than the independent Bernoulli process. Both processes have stationary mean 0.5, but only the DTMC has temporal dependence. This illustrates why a Markov model can represent clustering in a sequence that an independent Bernoulli model cannot.

2.2 Exercise 2: Estimating transitions from the attack trace

Objective: Estimate the transition probabilities of a 2-DTMC from the sequence of daily server attack indicators in `2dtmcdata.txt`.

Approach: The script `E2_AttackDTMC.R` imports the supplied file with the tab separator specified in the exercise. It counts each consecutive transition $i \to j$ and divides by the number of transitions originating in state i. The trace contains 2,000 observations and thus 1,999 transitions.

Table 1: Estimated transition counts and probabilities for the attack trace.

	Transition counts		Estimated probabilities	
Current state	To 0	To 1	To 0	To 1
0 (not attacked)	623	577	0.5192	0.4808
1 (attacked)	577	222	0.7222	0.2778

Results and conclusion: The estimated transition matrix, with rows denoting the current state, is

$$\widehat{P} = \begin{bmatrix} 0.5192 & 0.4808 \\ 0.7222 & 0.2778 \end{bmatrix}.$$

The fitted chain estimates an attack probability of approximately 0.3997 in steady state. An attack is followed by another attack with estimated probability 0.2778; otherwise, the process returns to the non-attacked state with probability 0.7222. These are empirical estimates from this finite trace, rather than known properties of all server attacks.

2.3 Exercise 3: Slotted ALOHA

Objective: Study the theoretical and simulated throughput of slotted ALOHA with a finite number of users, explain the effect of the parameters N, σ, and p, and compare ALOHA with carrier-sensing protocols.

2.3.1 a. Theoretical throughput

Approach: The supplied `aloha_plot.R` script uses `aloha_theo()` to plot theoretical throughput versus σ. With $N = 10$, it plots four curves for p: 0.3, 0.4, 0.5, and 0.6. With $N = 25$, it plots $p = 0.2$ and $p = 0.3$. In both plots, σ is logarithmically spaced as specified in the script.

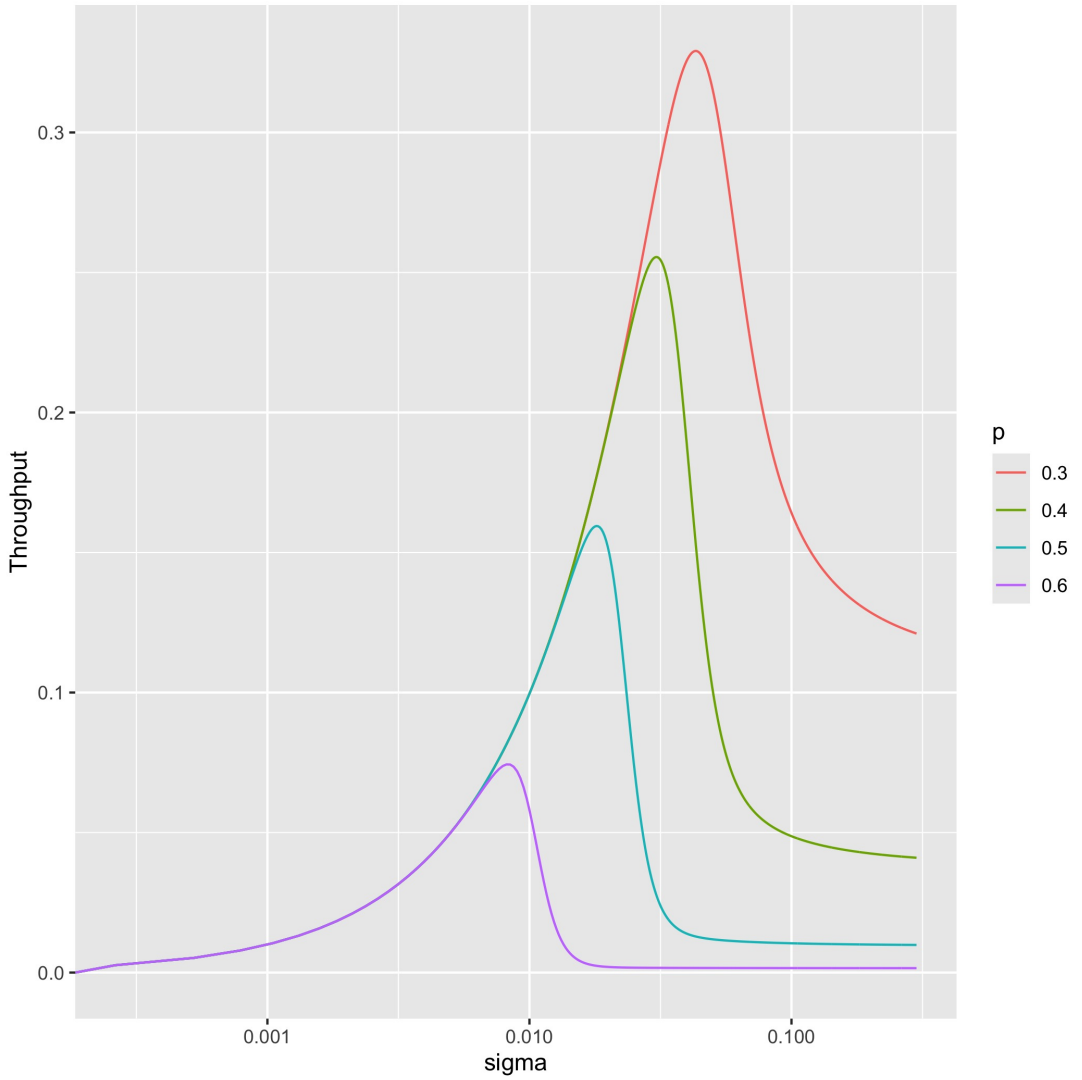


Figure 2: Theoretical slotted-ALOHA throughput for $N = 10$ and the four requested values of p.

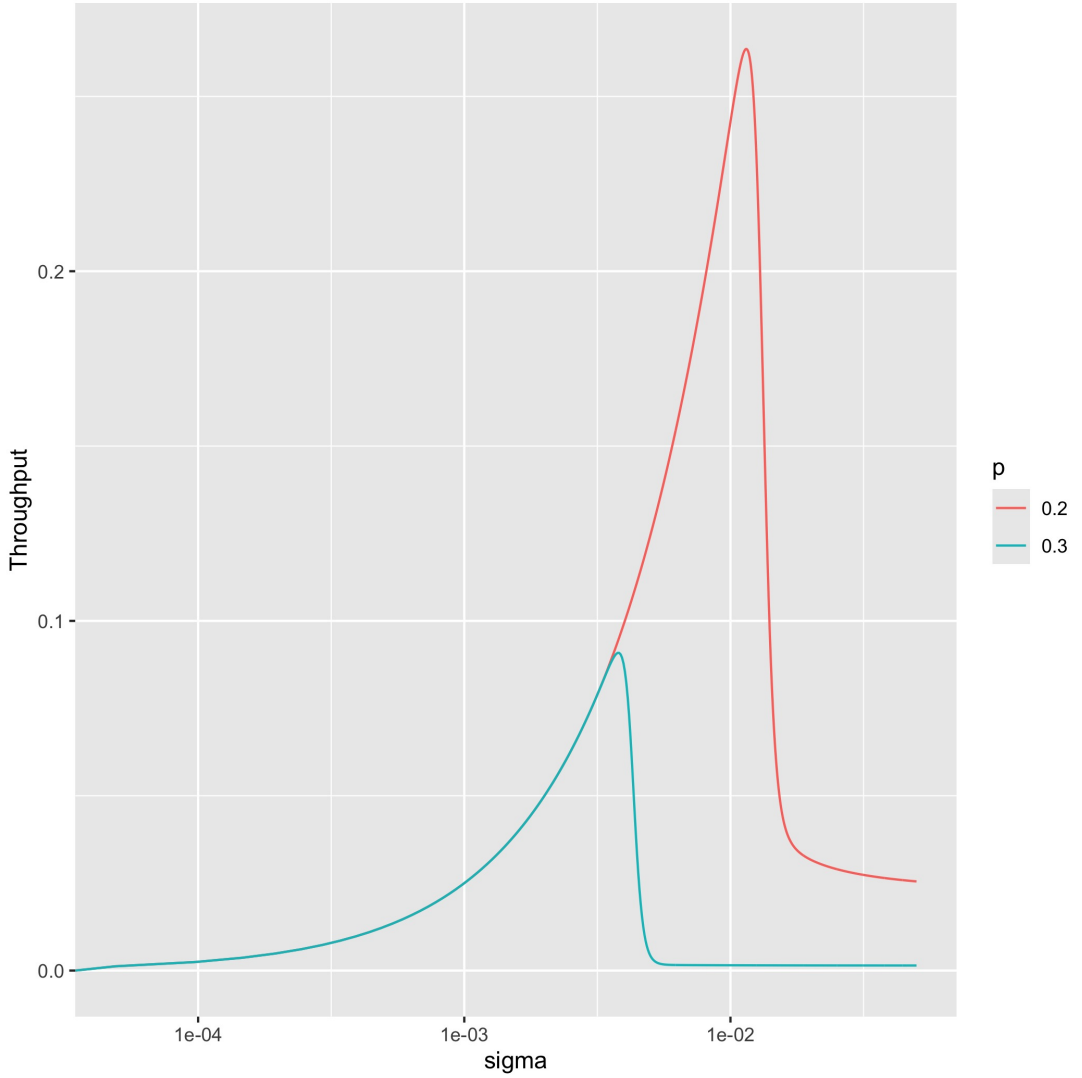


Figure 3: Theoretical slotted-ALOHA throughput for $N = 25$ and the two requested values of p.

Results and conclusion: Increasing σ makes more THINKING users transmit. Throughput can first increase as successful attempts become more frequent, then decrease when the added attempts create more collisions and BACKLOGGED users. Increasing p makes BACKLOGGED users retry more often; this can clear a small backlog, but can also increase collisions when many users are waiting. Increasing N adds potential transmissions and contention, so throughput depends on the interaction between N, p, and σ, rather than increasing monotonically with any one parameter.

2.3.2 b. Discrete-event simulation

Approach: Users are statistically identical, so the state is only the backlog size n. In each slot, $T \sim \text{Bin}(N - n, \sigma)$ THINKING and $B \sim \text{Bin}(n, p)$ BACKLOGGED users transmit. If $T + B = 1$ the slot is a success ($n \to n - 1$ if $B = 1$); otherwise $n \to n + T$, since colliding THINKING users become BACKLOGGED. Throughput is the fraction of successful slots. We

ran $R = 20$ independent replications of 2×10^5 slots (after 10^4 warm-up slots) and computed 95% Student-t confidence intervals. The theoretical values come from `aloha_theo()`.

Table 2: Theoretical and simulated throughput ($N = 10$; simulation: mean $\pm$ 95% CI half-width).

N	σ	p	Theory	Simulation	Rel. err. (%)	Theory in CI?
10	0.001	0.3	0.01000	0.01001 ± 0.00011	+0.09	yes
10	0.050	0.3	0.31541	0.31608 ± 0.00125	+0.21	yes
10	0.100	0.3	0.16415	0.16445 ± 0.00040	+0.18	yes
10	0.100	0.6	0.00160	0.00158 ± 0.00004	−1.10	yes

Conclusion: The theoretical value lies inside the confidence interval in all four cases. The relative errors are below 0.3% for the first three cases. The last one is about −1.1%, which is within the CI (its half-width is about 2% of the mean) because the throughput is so low that few successes are observed. The four operating points span two orders of magnitude, from light load ($\approx N\sigma$) to a collapsed system where most users are backlogged. Raising σ from 0.05 to 0.1 halves the throughput, and raising p from 0.3 to 0.6 at $\sigma = 0.1$ reduces it by a factor of about 100.

2.3.3 c. ALOHA vs. CSMA, CSMA/CD and CSMA/CA

Approach: Random-access throughput is limited by collisions. The protocols differ in (i) how long a frame is exposed to collisions (the *vulnerable period*) and (ii) how much channel time a collision wastes. Let T be the frame transmission time, τ the propagation delay and $a = \tau/T$.

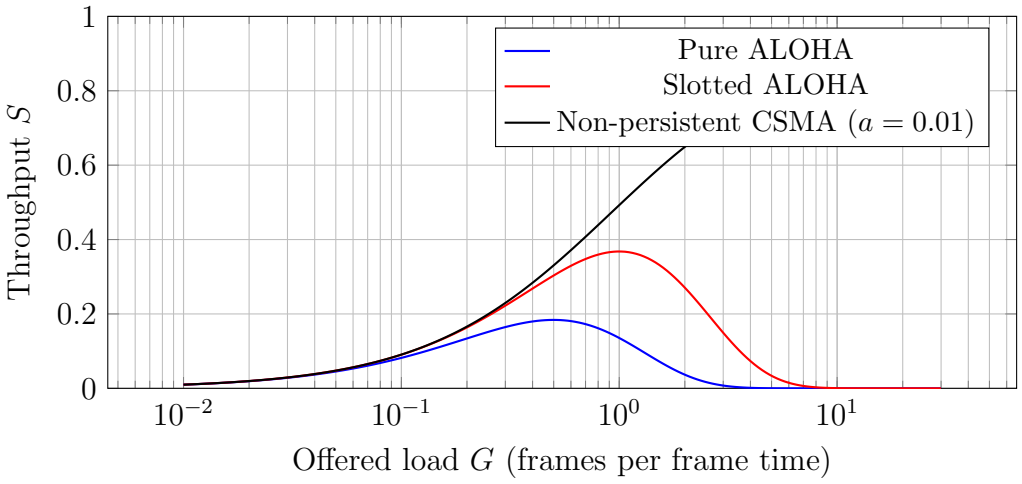


Figure 4: Classical throughput curves (infinite population).

Table 3: Random-access protocols compared.

Protocol	How it works	Throughput	Used in
Pure ALOHA	Transmit immediately; no ACK ⇒ retry after a random delay.	Vulnerable period $2T$. $S_{\max} = 1/(2e) \approx 0.18$.	ALOHAnet, LoRaWAN uplink
Slotted ALOHA	Same, but only at slot boundaries.	Vulnerable period T. $S_{\max} = 1/e \approx 0.37$.	RFID (EPC Gen2), cellular/satellite random-access channels
CSMA	"Listen before talk": if the channel is busy, defer (1-persistent: send when idle; non-persistent: wait a random time and re-sense; p-persistent: send with probability p per slot).	Vulnerable period shrinks to τ. For $a = 0.01$, non-persistent reaches ≈ 0.8; falls as a grows.	Early packet radio; basis of the two variants below
CSMA/CD	CSMA plus listening *while* sending. On a collision: abort, send a jam signal, wait a random time (binary exponential backoff). Needs frame time $\geq 2\tau$.	A collision wastes $\approx 2\tau$ instead of T. Efficiency $\approx 1/(1+5a)$ (approx.).	Shared Ethernet (IEEE 802.3, hubs/coax); not in switched full-duplex
CSMA/CA	Wait for the channel to be idle (DIFS), then count down a random backoff, frozen while busy; receiver ACKs; optional RTS/CTS. Avoids collisions since a radio cannot detect them while sending.	Fewer collisions than CSMA, but backoff and ACK overhead lower the gain.	IEEE 802.11 Wi-Fi, IEEE 802.15.4 (Zigbee)

Conclusion: Each step wastes less channel time. Slotting halves the vulnerable period ($2T \to T$) and doubles the peak throughput. Carrier sensing cuts it to about τ, so for $a \ll 1$ throughput exceeds 0.8. Collision detection reduces the cost of a collision from a full frame to about 2τ. CSMA/CA gives up part of this gain to work where collisions cannot be detected (wireless). When a is large (e.g. satellite links), sensing is of little help and ALOHA-type access remains in use. The simulated peaks in Section X agree with the slotted-ALOHA limit $1/e$.

References: Bertsekas & Gallager, *Data Networks*, Ch. 4; Rom & Sidi, *Multiple Access Protocols*; Kleinrock & Tobagi (1975); Kurose & Ross, *Computer Networking*.

3 B. Discrete-time Markov chains and Continuous-time Markov chains

3.1 Exercise 4: Limiting probabilities of a DTMC

Approach:

- **(i) Balance equations:** solve $\pi P = \pi$, $\sum_i \pi_i = 1$ as the linear system $(P^T - I)\pi = 0$, with its last equation replaced by $\sum_i \pi_i = 1$ (`solve()`).
- **(ii) Matrix power:** compute P^n with `expm` (`P %^% n`). For large n all rows converge to π.
- **(iii) Simulation:** starting in state 0, each step draws the next state with `sample()` using the row of P of the current state. π_i is the fraction of time in state i after a warm-up of 10^3 steps; we ran 30 independent runs of 10^5 steps and computed 95% Student-t confidence intervals.

Results:

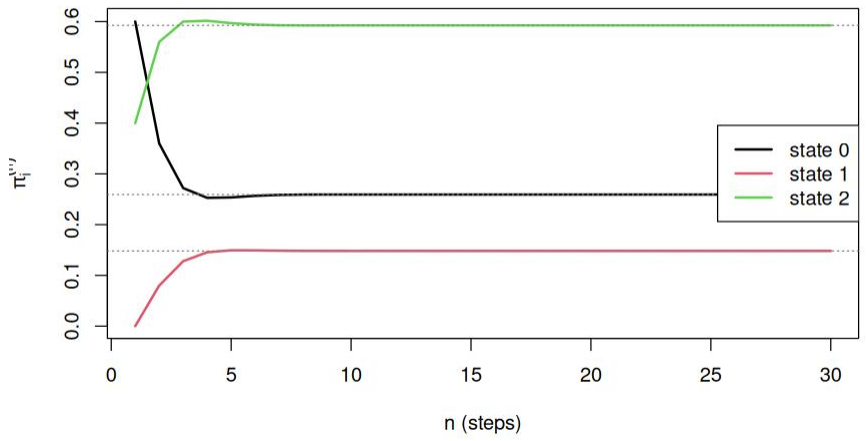


Figure 5: First row of P^n versus n. Dotted lines: solution of the balance equations.

Table 4: Limiting state probabilities (simulation: mean $\pm$ 95% CI half-width).

Method	π_0	π_1	π_2
Balance equations	0.2593	0.1481	0.5926
Matrix power ($n = 50$)	0.2593	0.1481	0.5926
Simulation	0.2597 ± 0.0007	0.1479 ± 0.0002	0.5924 ± 0.0008

Conclusion: The three methods agree. The balance equations give $\pi_1 = \frac{4}{7}\pi_0$ and $\pi_2 = 4\pi_1$, hence $\pi = (7, 4, 16)/27$. The rows of P^n converge to π after about 10 steps (Figure 5), because the second-largest eigenvalue modulus of P is ≈ 0.37, so the error shrinks as 0.37^n; at $n = 50$ all rows equal π to within 10^{-15}. The simulation estimates are within 0.2% of the exact values. The confidence intervals of π_0 and π_2 contain the exact values, while that of π_1 narrowly misses it. This is expected: with three 95% intervals, one miss is not unusual, and the discrepancy is negligible in practice. State 2 dominates ($\approx 59\%$) because it is easy to enter (from states 0 and 1) and hard to leave (self-loop 0.8), while state 1 is the rarest: it is only entered from state 2 with probability 0.2.

3.2 Exercise 5: Simulation of a CTMC

Approach: The chain of Figure 2 has generator matrix

$$Q = \begin{bmatrix} -2 & 2 & 0 \\ 1 & -2 & 1 \\ 1 & 1 & -2 \end{bmatrix}.$$

Theory: the limiting probabilities solve $\pi Q = 0$, $\sum_i \pi_i = 1$, as a linear system in which the last equation is replaced by $\sum_i \pi_i = 1$ (`solve()`). *Simulation:* the CTMC was simulated in two equivalent ways.

- **View 1:** the chain stays in state i for an exponential time with rate $q_i = \sum_{j \neq i} \lambda_{ij}$, then jumps to state j with probability λ_{ij}/q_i.
- **View 2:** each possible transition $i \to j$ has an independent exponential timer with rate λ_{ij}; the smallest timer fires, and the chain moves to the corresponding state.

In both cases π_i is estimated as the fraction of time spent in state i. We ran 30 independent runs of 10^5 jumps each (after 10^3 warm-up jumps), starting in state 1, and computed 95% Student-t confidence intervals.

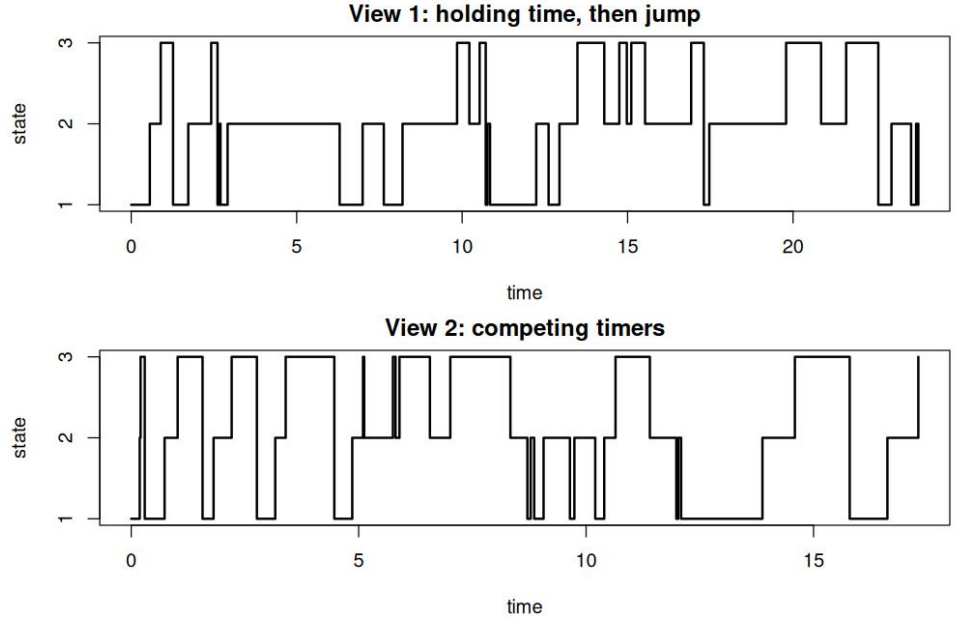


Figure 6: Sample paths of the CTMC generated with each view (first 40 jumps).

Table 5: Limiting state probabilities (simulation: mean ± 95% CI half-width).

Method	π_1	π_2	π_3
Theory ($\pi Q = 0$)	0.3333	0.4444	0.2222
Simulation, view 1	0.3333 ± 0.0007	0.4444 ± 0.0006	0.2222 ± 0.0006
Simulation, view 2	0.3333 ± 0.0007	0.4446 ± 0.0005	0.2220 ± 0.0007

Conclusion: Solving $\pi Q = 0$ gives $\pi_3 = \pi_2/2$ and $\pi_1 = \frac{3}{2}\pi_3$, hence $\pi = (3, 4, 2)/9$. Both views reproduce this result, with the exact values inside the confidence intervals. This is expected, since the two views describe the same process: the minimum of independent exponentials with rates λ_{ij} is exponential with rate q_i, and the winning timer is j with probability λ_{ij}/q_i. Here all states have the same exit rate $q_i = 2$ (mean holding time 0.5), so the fraction of time in each state equals the fraction of visits to it. State 2 is the most frequent because it is the only exit of state 1 and is reachable from both other states, while state 3 is entered only from state 2, and only half of the time.

4 C. Queuing systems

4.1 Exercise 6: Stability of the M/M/1 queue

Approach: We simulated an M/M/1 queue and measured the average queuing delay over the first `NumQueueCompleted` customers served (including those with zero delay), for three run lengths: 10^3, 10^4 and 10^5. Loads: $\rho = 1$ ($\lambda = \mu = 1$) and $\rho = 2$ ($\lambda = 2$, $\mu = 1$), plus a stable reference $\rho = 0.5$ ($\lambda = 1$, $\mu = 2$), whose theoretical delay is $\rho/(\mu - \lambda) = 0.5$. Each point is the mean of 20 independent runs, with 95% Student-t confidence intervals. No warm-up was used since the purpose is to observe the transient growth.

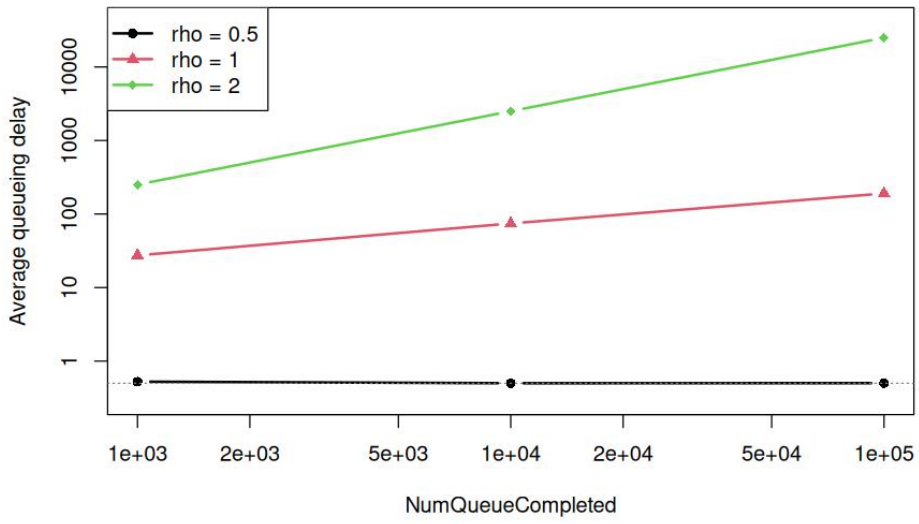


Figure 7: Average queuing delay versus run length (log-log). Dotted line: theoretical delay for $\rho = 0.5$.

Table 6: Average queuing delay (mean ± 95% CI half-width, 20 runs).

`NumQueueCompleted`	$\rho = 0.5$	$\rho = 1$	$\rho = 2$
10^3	0.525 ± 0.037	27.4 ± 8.3	251 ± 12
10^4	0.500 ± 0.019	74.5 ± 17.6	2503 ± 28
10^5	0.502 ± 0.005	190.6 ± 48.4	24980 ± 94

Conclusion: For $\rho = 0.5$ the estimate converges to the theoretical value 0.5 as the run grows, so the system has a steady state. For $\rho \geq 1$ it does not: the estimate keeps increasing with the run length, so the system is *not stable*. At $\rho = 1$ the delay grows roughly as $\sqrt{n}$ (by a factor of about 2.6 per decade of n, compatible with $\sqrt{10} \approx 3.2$ given the confidence intervals), and the relative width of the interval stays around 25–30%, so the estimate does not converge to a long-run average. At $\rho = 2$ the queue grows without bound and the delay grows linearly in n: the values 251, 2503 and 24980 agree with the prediction $n/4$ for $\lambda = 2$, $\mu = 1$. The confidence intervals are narrow, but the estimates are not long-run averages, since they just reflect the run length. Therefore steady-state performance measures only make sense for $\rho < 1$.

4.2 Exercise 7: Point-to-point link (`ppl.R`)

Setup: A link with $C = 100$ kb/s serves two Poisson flows with 1000-bit packets, so $\mu = 100$ pkt/s and the mean transmission time is $S = 10$ ms. Only `parameters.R` was changed. The simulator reports the average delay in the system (queuing plus transmission) of each flow.

4.2.1 a. Poisson arrivals, exponential packet sizes

Approach: Both flows have the same priority and share one FIFO queue. The sum of two Poisson flows is Poisson and the sizes are exponential, so this is an M/M/1 queue with $\lambda = \lambda_1 + \lambda_2$ and $W = 1/(\mu - \lambda)$, equal for both flows.

Conclusion: The delay grows seven-fold when ρ goes from 0.3 to 0.9, as predicted by $W = 1/(\mu - \lambda)$. At light load the simulation is within 4% of theory. Both flows have the same theoretical delay (FIFO does not distinguish them), so the gap between their estimates (14.47 vs. 14.84 ms) shows the level of simulation noise. Throughput equals the offered load (9985 and 20199 vs. 10000 and 20000 bit/s), as expected in a stable system.

4.2.2 b. Poisson arrivals, fixed packet sizes

Approach: Same arrival rates and mean packet size, but every service time equals $S = 10$ ms, so this is an M/D/1 queue: $W_q = \dfrac{\rho S}{2(1-\rho)}$ and $W = S + W_q$.

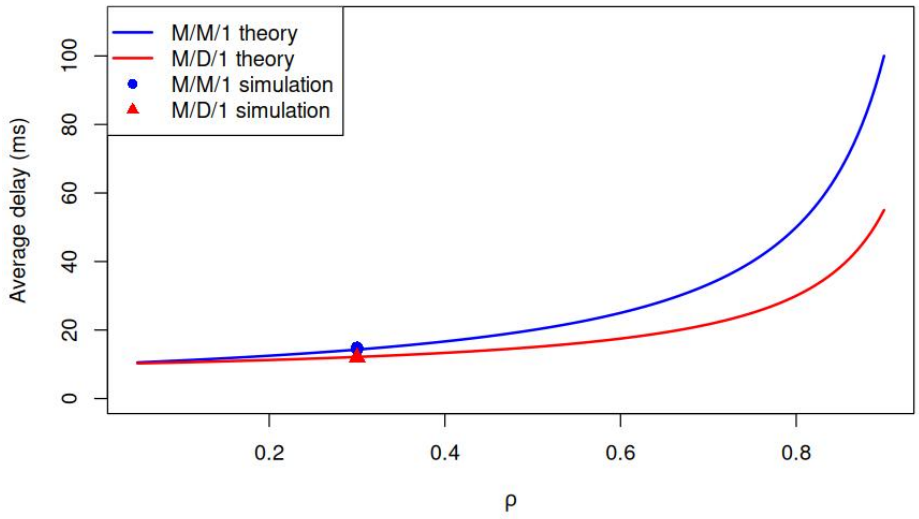


Figure 8: Average delay versus load: theory (lines) and simulation (points).

Exponential vs. fixed sizes: For the same λ and mean packet size, fixed sizes give a lower delay: 12.1 vs. 14.3 ms at $\rho = 0.3$ (-15%) and 55 vs. 100 ms at $\rho = 0.9$ (-45%). The simulations confirm the ordering (Figure 8).

Impact of the service-time distribution: By the Pollaczek–Khinchine formula, $W_q = \dfrac{\rho S(1 + c_S^2)}{2(1 - \rho)}$, where c_S^2 is the squared coefficient of variation of the service time: $c_S^2 = 1$ for exponential and 0 for fixed sizes. Hence the M/D/1 queuing delay is exactly half of the M/M/1 one. The average delay depends on the variability of the service time and not only on its mean, and the effect grows as $\rho \to 1$.

4.2.3 c. Strict priority scheduling

Approach: Packets are never interrupted, so this is a non-preemptive M/D/1 queue with priorities. Cobham's formula gives, for class k,

$$W_k = S + \frac{W_0}{(1 - \sigma_{k-1})(1 - \sigma_k)}, \qquad W_0 = \sum_i \frac{\lambda_i S^2}{2}, \qquad \sigma_k = \sum_{i \le k} \rho_i.$$

With $\lambda_1 = 10$ (priority 1) and $\lambda_2 = 85$ pkt/s (priority 2), $\rho = 0.95$ and $W_0 = 4.75$ ms, so $W_1 = 15.28$ ms and $W_2 = 115.56$ ms, a ratio of 7.6. The default rates (10 and 80) give a ratio of exactly 4.0, too close to the required limit.

Conclusion: Priorities separate the two flows strongly. The high-priority flow only waits for the packet in transmission and for packets of its own class ($W_q = 5.3$ ms), while the low-priority flow absorbs the load ($W_2 \approx 7.6\, W_1$). A FIFO queue at the same load would give both flows 105 ms. The λ-weighted mean of the priority delays, $(10 \cdot 15.28 + 85 \cdot 115.56)/95 = 105$ ms, is the same: priorities redistribute the delay between flows but do not reduce it.

5 D. Workloads

6 Conclusion

The three class 5 exercises demonstrate how DTMCs extend independent Bernoulli models by representing temporal dependence, how transition probabilities can be estimated directly from observed state changes, and how the stationary behavior of a contention protocol can be studied analytically and by simulation. The estimated and simulated results are consistent with the corresponding models, while also showing that finite data and operating parameters matter when interpreting performance.